# FULL-STACK DEVELOPMENT LABORATORY RECORD

*A Laboratory Record submitted in fulfillment of the requirements for the course*

***Full-Stack Development Laboratory***

**BACHELOR OF TECHNOLOGY**

**IN**

**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING(AI&ML)**

*Submitted by*


## V. Govardhan

**(A24126552186)**


*Under the guidance of*

**Mrs. Ch. Swathi**

*Assistant Professor*




# ANITS


**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING(AI&ML)**

**ANIL NEERUKONDA INSTITUTE OF TECHNOLOGY AND SCIENCES**

**(UGC AUTONOMOUS)**

**(Permanently Affiliated to AU, Approved by AICTE and Accredited by NBA&NAAC with 'A+' Grade)**

Sangivalasa, Bheemili Mandal, Visakhapatnam Dist. (A.P)


**2026-2027**

# ANIL NEERUKONDA INSTITUTE OF TECHNOLOGY AND SCIENCES

**(UGC AUTONOMOUS)**

(Affiliated to AU, Approved by AICTE and Accredited by NAAC)

SANGIVALASA, Bheemili Mandal, VISAKHAPATNAM-531162




# ANITS

## CERTIFICATE

This is to certify that the laboratory record entitled "FULL-STACK DEVELOPMENT LABORATORY" submitted by **V. Govardhan** (Regd. No. A24126552186) in partial fulfillment of the requirements for the Full-Stack Development Laboratory course in the Department of Computer Science and Engineering (AI&ML) during the year 2026-2027 of Anil Neerukonda Institute of Technology and Sciences (A+), Visakhapatnam is a record of bonafide work carried out under my guidance and supervision.

| | | |
|---|---|---|
| **Mrs. Ch. Swathi**<br>**Assistant Professor**<br>**(Faculty In-Charge)** | **Head of the Department**<br>**CSE (AI&ML)**<br>**ANITS** | **External Examiner**<br><br>**Date: _____** |

# ACKNOWLEDGEMENT

I express my deep gratitude and sincere respect to the Head of the Department of Computer Science and Engineering (AI&ML) and to the Management of Anil Neerukonda Institute of Technology and Sciences (ANITS) for providing well-equipped laboratory facilities, a disciplined academic environment and constant encouragement throughout this academic year.

I am deeply thankful to my faculty guide and course instructor, **Mrs. Ch. Swathi**, Assistant Professor, whose clear explanations, regular reviews and patient guidance helped me to understand every concept and to complete all ten experiments and the MERN stack project. Her suggestions on writing clean code and documenting each step carefully were of great value to me.

I convey my sincere thanks to all the faculty members of the department for their support and for the strong foundation in programming, databases and machine learning that made it easier to take up this laboratory course.

I also thank the laboratory technical staff for maintaining the systems and the software required for the experiments, and for helping me whenever I faced a technical problem during the laboratory sessions.

My heartfelt thanks go to my classmates and friends for their cooperation, helpful technical discussions and encouragement, and to my parents for their constant support and motivation, without which this work would not have been possible.

The knowledge and practical experience gained in this laboratory will be very useful in my future studies, projects and career.

**V. Govardhan**
Regd. No: A24126552186
B.Tech CSE (AI&ML)
ANITS, Visakhapatnam

# TABLE OF CONTENTS / INDEX


| S.No. | Experiment Title | Course Outcome | Page No. | Grade | Faculty Signature |
|---|---|---|---|---|---|
| 1 | Design Static Webpage Using HTML Components | CO1 | 5-8 | | |
| 2 | Design Webpage Using HTML5 and CSS3 | CO1 | 9-14 | | |
| 3 | JavaScript Program: Arrays and Functions | CO1 | 15-18 | | |
| 4 | Develop a Dynamic Webpage Using JavaScript | CO1 | 19-22 | | |
| 5 | Implement Modules in Node.js | CO2 | 23-25 | | |
| 6 | Develop a Mini Application Using Express and Node.js | CO2 | 26-30 | | |
| 7 | Create Functional and Class Components Using React | CO3 | 31-35 | | |
| 8 | Implement HTTP Methods Using JSON Server | CO3 | 36-41 | | |
| 9 | Develop CRUD Operations Using MongoDB | CO4 | 42-46 | | |
| 10 | Main Project: Library Book Manager Using MERN Stack | CO4 | 47-52 | | |

# EXPERIMENT 1
# DESIGN STATIC WEBPAGE USING HTML COMPONENTS

*[Course Outcome: CO1]*

## AIM:

To design a static student-profile webpage using basic HTML components: headings, paragraphs, text formatting, lists, links, a table and a form with several input types.

## THEORY:

HTML (HyperText Markup Language) describes the structure of a webpage with elements written as tags. A static page shows the same fixed content to every visitor and does not react to the user. The browser reads the markup from top to bottom and builds a tree of elements, which it then draws on the screen.

## IMPORTANT CONCEPTS:

- **Document skeleton:** `<!DOCTYPE html>` tells the browser to use standard mode, `<html>` is the root, `<head>` holds information that is not displayed (character set, title) and `<body>` holds everything visible.
- **Block and inline elements:** `<h1>`, `<p>`, `<ul>`, `<table>` and `<form>` are block elements that start on a new line; `<a>`, `<b>`, `<i>` and `<input>` are inline and stay within a line.
- **Lists and tables:** `<ul>` gives bullets, `<ol>` gives numbers and each entry is an `<li>`. In a table `<tr>` is a row, `<th>` a heading cell and `<td>` a data cell.
- **Form controls:** `type` selects a text box, email box or radio button; `<select>` gives a drop-down list; `<textarea>` accepts several lines; `name` identifies each value when the form is submitted.
- **Entity codes:** characters such as & and < must be written as `&` and `<` inside the markup.

| Element | Purpose | Example |
|---|---|---|
| <h1> to <h6> | Headings, h1 is the most important | <h1>Student Profile</h1> |
| <ul> / <ol> | Bulleted and numbered lists | <li>Python</li> |
| <table> | Rows and columns of data | <tr><td>Monday</td></tr> |
| <input> | Single-line user input | <input type="email"> |
| <select> | Drop-down choice | <option>CSE</option> |

## PROCEDURE:

1. Create a folder for the lab and a new file named exp1.html in a text editor such as VS Code.
2. Type the skeleton: DOCTYPE, html, head with the title, and body.
3. Add the headings and paragraphs, then the unordered list, the ordered list, the timetable table and the links.
4. Add the feedback form with a text box, an email box, radio buttons, a drop-down and a text area.
5. Save the file, open it in a browser and compare every element with the source code.

## SOURCE CODE:

File Name: exp1.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Student Profile Page</title>
</head>
<body>
  <h1>Student Profile</h1>
  <h3>Full-Stack Development Laboratory - Experiment 1</h3>
  <p>This page is built only with basic HTML components: headings, paragraphs, lists,
      links, a table and a form.</p>

  <h2>About Me</h2>
  <p>I am <b>V. Govardhan</b>, a second-year B.Tech student of <i>CSE
      (AI&amp;amp;ML)</i> at ANITS, Visakhapatnam.</p>

  <h2>Skills I Am Learning</h2>
  <ul>
    <li>HTML and CSS</li>
    <li>JavaScript</li>
    <li>Python for Machine Learning</li>
  </ul>

  <h2>Semester Plan</h2>
  <ol>
    <li>Complete the web technologies experiments</li>
    <li>Build a small project</li>
    <li>Prepare for the lab viva</li>
  </ol>

  <h2>Weekly Timetable</h2>
  <table border="1" cellpadding="6">
    <tr><th>Day</th><th>Subject</th><th>Time</th></tr>
    <tr><td>Monday</td><td>Machine Learning</td><td>9:00 AM</td></tr>
```

File Name: exp1.html (continued)

```
    <tr><td>Wednesday</td><td>Full-Stack Lab</td><td>2:00 PM</td></tr>
    <tr><td>Friday</td><td>Data Structures</td><td>11:00 AM</td></tr>
  </table>

  <h2>Useful Links</h2>
  <p><a href="https://developer.mozilla.org">MDN Web Docs</a> | <a
      href="https://react.dev">React Documentation</a></p>

  <h2>Feedback Form</h2>
  <form>
    <label>Name: <input type="text" name="name"></label><br><br>
    <label>Email: <input type="email" name="email"></label><br><br>
    Year:
    <label><input type="radio" name="year" value="2"> 2nd</label>
    <label><input type="radio" name="year" value="3"> 3rd</label><br><br>
    <label>Branch:
      <select name="branch">
        <option>CSE (AI&amp;amp;ML)</option>
        <option>CSE</option>
        <option>IT</option>
      </select>
    </label><br><br>
    <label>Comments:<br><textarea name="comments" rows="3"
        cols="30"></textarea></label><br><br>
    <button type="submit">Submit</button>
  </form>
</body>
</html>
```

## OUTPUT:

The browser shows the main heading, a short introduction, a bulleted list of skills, a numbered semester plan, a three-row timetable table, two links and a feedback form containing every control.

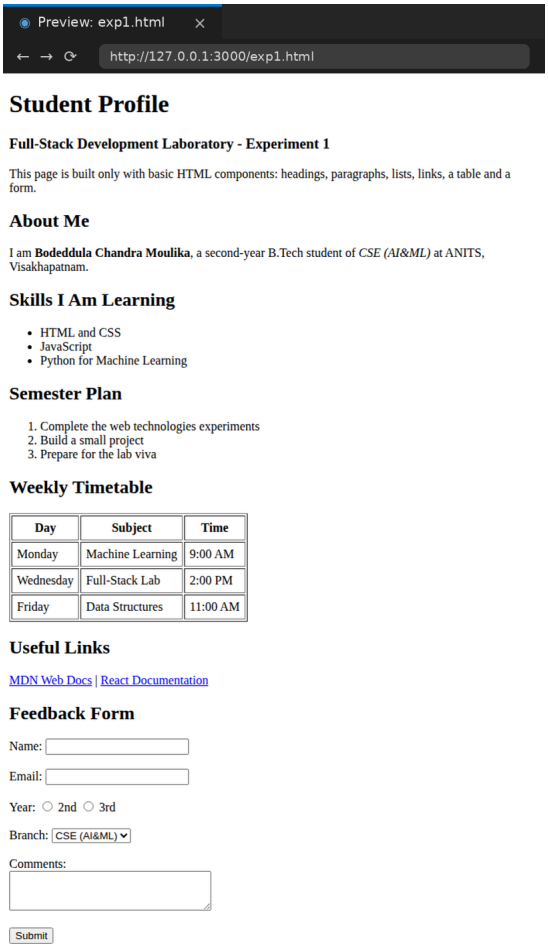



*Figure 1: Student profile page rendered in the browser*

## RESULT:

A static webpage was designed using HTML components and verified in the browser.

## VIVA QUESTIONS:

**Q1. What does the DOCTYPE declaration do?**

Ans: It tells the browser that the page is written in HTML5 so that it renders in standard mode instead of quirks mode.

**Q2. What is the difference between `<ul>` and `<ol>`?**

Ans: `<ul>` shows items with bullets when order does not matter; `<ol>` numbers the items when order matters.

**Q3. Why does every form control need a `name` attribute?**

Ans: The name is the key under which the control's value is sent when the form is submitted; controls without a name are not sent.

**Q4. How do `<b>` and `<strong>` differ?**

Ans: Both look bold, but `<strong>` marks text as important and is read with emphasis by screen readers, while `<b>` is only visual styling.

# EXPERIMENT 2
# DESIGN WEBPAGE USING HTML5 & CSS3

*[Course Outcome: CO1]*

## AIM:

To design a responsive cafe webpage using HTML5 semantic elements and style it with an external CSS3 stylesheet that uses variables, Grid, Flexbox, transitions and a media query.

## THEORY:

HTML5 semantic elements such as `<header>`, `<nav>`, `<main>`, `<section>`, `<article>`, `<aside>` and `<footer>` describe the role of each part of a page. CSS3 controls colour, fonts, layout and responsiveness. An external stylesheet keeps presentation separate from content and lets many pages share the same design.

## IMPORTANT CONCEPTS:

- **CSS variables:** values such as `--brand` are declared once in `:root` and reused with `var(--brand)`, so a theme can be changed in one place.
- **CSS Grid:** `display: grid` with `grid-template-columns: repeat(3, 1fr)` creates three equal columns for the menu cards.
- **Flexbox:** `display: flex` with `justify-content` and `gap` lays the navigation links in a row.
- **Transitions and hover:** `transition` animates a change smoothly, for example a card moving up by 5 px on `:hover`.
- **Box model:** content, padding, border and margin; `box-sizing: border-box` makes width include padding and border.
- **Media query:** `@media (max-width: 600px)` switches the grid to one column and stacks the navigation on narrow screens.

| CSS feature | Used for |
|---|---|
| linear-gradient() | Gradient background of the header |
| grid-template-columns | Three-column menu layout |
| transition / transform | Smooth hover lift on cards |
| @media (max-width: 600px) | Responsive single-column layout |

## PROCEDURE:

1. Create a folder exp2 and add two files: index.html and style.css.
2. Write the HTML structure in the order header, nav, main (two sections), aside and footer.
3. Define the colour variables in :root, then style body, header, nav, cards, aside and footer.
4. Open index.html in a browser and hover over a card and a navigation link.
5. Reduce the window width below 600 px and confirm that the cards and the navigation stack vertically.

## SOURCE CODE:

File Name: index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Campus Cafe - HTML5 and CSS3</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <header>
    <h1>Campus Cafe</h1>
    <p>Fresh snacks between your lab sessions</p>
  </header>
  <nav>
    <a href="#menu">Menu</a>
    <a href="#offers">Offers</a>
    <a href="#visit">Visit Us</a>
  </nav>
  <main>
    <section id="menu">
      <h2>Today's Menu</h2>
      <div class="grid">
        <article class="card"><h3>Masala Dosa</h3><p>Crisp dosa with potato
            filling.</p><span class="price">Rs. 50</span></article>
        <article class="card"><h3>Veg Sandwich</h3><p>Grilled with fresh
            vegetables.</p><span class="price">Rs. 40</span></article>
        <article class="card"><h3>Filter Coffee</h3><p>Strong South Indian
            coffee.</p><span class="price">Rs. 20</span></article>
      </div>
    </section>
    <section id="offers">
      <h2>Offers</h2>
      <p>Show your ID card and get <strong>10% off</strong> on every order before 11
          AM.</p>
```

File Name: index.html (continued)

```
    </section>
  </main>
  <aside>
    <p>Tip: Resize the window to see the cards stack on small screens.</p>
  </aside>
  <footer id="visit">
    <p>Block C, ANITS Campus &bull; Designed by V. Govardhan</p>
  </footer>
</body>
</html>
```

File Name: style.css

```
:root {
  --brand: #8e44ad;
  --dark: #2d1b3d;
  --accent: #f39c12;
  --light: #faf6fc;
}
* { box-sizing: border-box; }
body {
  font-family: "Segoe UI", Arial, sans-serif;
  margin: 0;
  background: var(--light);
  color: #333;
}
header {
  background: linear-gradient(135deg, var(--brand), var(--dark));
  color: #fff;
  text-align: center;
  padding: 30px 20px;
}
header p { margin: 5px 0 0; opacity: 0.85; }
nav {
  background: var(--dark);
  display: flex;
  justify-content: center;
  gap: 30px;
  padding: 12px;
}
nav a {
  color: #fff;
  text-decoration: none;
  transition: color 0.3s;
}
nav a:hover { color: var(--accent); }
main { padding: 20px; max-width: 900px; margin: auto; }
```

File Name: style.css (continued)

```
.grid {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 15px;
}
.card {
  background: #fff;
  padding: 15px;
  border-radius: 10px;
  box-shadow: 0 3px 8px rgba(0, 0, 0, 0.12);
  transition: transform 0.3s;
}
.card:hover { transform: translateY(-5px); }
.price { color: var(--brand); font-weight: bold; }
section { margin-bottom: 25px; }
aside {
  max-width: 860px;
  margin: 0 auto 20px;
  padding: 10px 15px;
  background: #fdf2e0;
  border-left: 5px solid var(--accent);
}
footer {
  background: var(--dark);
  color: #fff;
  text-align: center;
  padding: 12px;
}
@media (max-width: 600px) {
  .grid { grid-template-columns: 1fr; }
  nav { flex-direction: column; align-items: center; gap: 8px; }
}
```

## OUTPUT:

At desktop width the page shows a purple gradient header, a navigation bar, three menu cards in a row, an offers section, a tip box with an orange left border and a dark footer. Below 600 px the cards and links stack in a single column.

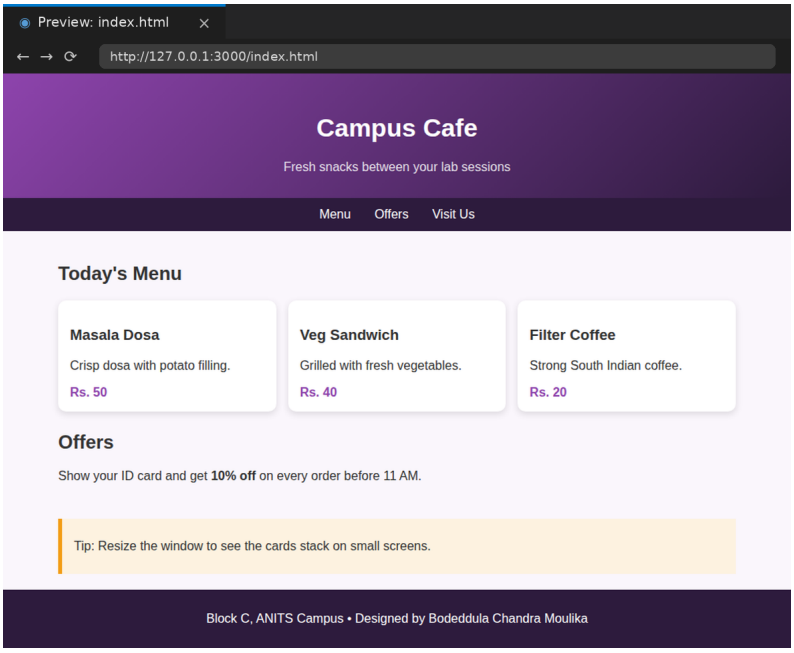



*Figure 2(a): Cafe page at desktop width (1000 px)*

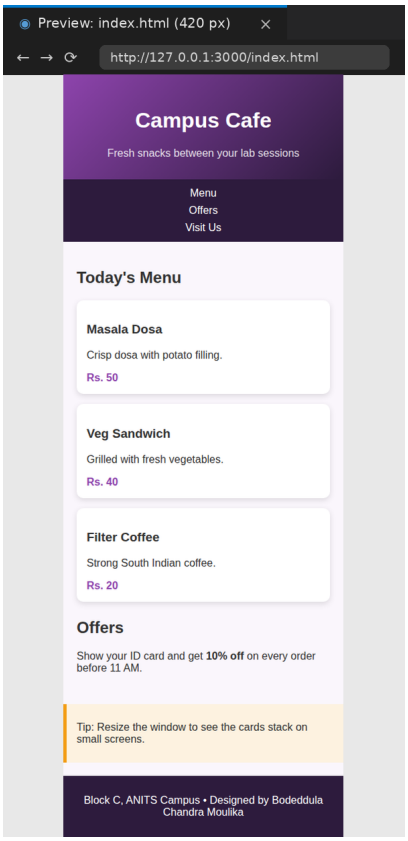



*Figure 2(b): Same page at 420 px width, single-column layout*

## RESULT:

A responsive webpage was designed using HTML5 semantic elements and styled with CSS3.

## VIVA QUESTIONS:

**Q1. What is the advantage of CSS variables?**

Ans: A value declared once can be reused everywhere, so changing a theme colour needs only one edit.

**Q2. How is Grid different from Flexbox?**

Ans: Flexbox arranges items along one axis (a row or a column); Grid arranges items in rows and columns at the same time.

**Q3. What does `box-sizing: border-box` do?**

Ans: It makes the declared width and height include padding and border, which makes sizing predictable.

**Q4. When is a media query applied?**

Ans: Only when its condition is true, such as the viewport being 600 px wide or narrower.

# EXPERIMENT 3
# WRITE A PROGRAM USING JAVASCRIPT (ARRAYS, FUNCTIONS)

*[Course Outcome: CO1]*

## AIM:

To write a JavaScript program that analyses student marks using arrays, objects, user-defined functions, recursion and the higher-order array methods.

## THEORY:

An array stores an ordered list of values that are read by index. JavaScript gives arrays powerful methods such as `map()`, `filter()`, `reduce()` and `sort()`. A function is a reusable block of code that can take parameters and return a value; a function may also call itself, which is called recursion.

## IMPORTANT CONCEPTS:

- **Array of objects:** each student is an object `{ name, marks }`; `students.map(s => s.marks)` extracts one field into a new array.
- **reduce():** combines all elements into a single value, used here to find the sum before dividing for the average.
- **sort():** `sort((a, b) => b.marks - a.marks)` orders the list from highest to lowest; the spread `[...students]` sorts a copy so the original is kept.
- **Recursion:** `factorial(n)` calls `factorial(n - 1)` until it reaches the base case `n <= 1`.
- **String handling:** a palindrome check removes non-alphanumeric characters with a regular expression, then compares the text with its reverse.

| Method | Purpose | Changes original? |
|---|---|---|
| map() | Transform every element into a new array | No |
| filter() | Keep elements that pass a test | No |
| reduce() | Combine all elements into one value | No |
| sort() | Order the elements | Yes |
| forEach() | Run a function for each element | No |

## PROCEDURE:

1. Create a file exp3.js in a new folder.
2. Write the functions average, highestAndLowest, gradeOf, factorial, fibonacci and isPalindrome.
3. Create the students array and call every function from the main part of the program.
4. Run the program with node exp3.js.
5. Compare the printed values with a hand calculation, for example the average (78+92+45+66+84)/5 = 73.

## SETUP AND RUN:

Commands

```
node exp3.js
```

## SOURCE CODE:

File Name: exp3.js

```
// Student marks analysis using arrays and functions
function average(arr) {
  return arr.reduce((sum, m) => sum + m, 0) / arr.length;
}

function highestAndLowest(arr) {
  return { highest: Math.max(...arr), lowest: Math.min(...arr) };
}

function gradeOf(mark) {
  if (mark >= 90) return "O";
  if (mark >= 80) return "A+";
  if (mark >= 70) return "A";
  if (mark >= 60) return "B";
  if (mark >= 50) return "C";
  return "F";
}

function factorial(n) {
  return n <= 1 ? 1 : n * factorial(n - 1);
}

function fibonacci(count) {
  const series = [0, 1];
  for (let i = 2; i < count; i++) {
    series.push(series[i - 1] + series[i - 2]);
  }
  return series.slice(0, count);
}

function isPalindrome(text) {
  const clean = text.toLowerCase().replace(/[^a-z0-9]/g, "");
  return clean === clean.split("").reverse().join("");
}
```

File Name: exp3.js (continued)

```
// Main program
const students = [
  { name: "Anil", marks: 78 },
  { name: "Bhavana", marks: 92 },
  { name: "Charan", marks: 45 },
  { name: "Divya", marks: 66 },
  { name: "Eswar", marks: 84 }
];
const marks = students.map(s => s.marks);

console.log("Marks:", marks);
console.log("Average:", average(marks).toFixed(2));
console.log("Highest / Lowest:", highestAndLowest(marks));

console.log("\nGrade report:");
students.forEach(s => console.log(`${s.name.padEnd(8)} ${s.marks}
    ${gradeOf(s.marks)}`));

const passed = students.filter(s => s.marks >= 50).map(s => s.name);
console.log("\nPassed students:", passed);

const ranked = [...students].sort((a, b) => b.marks - a.marks);
console.log("Rank list:", ranked.map((s, i) => `${i + 1}.${s.name}`).join(", "));

console.log("\nFactorial of 6:", factorial(6));
console.log("First 8 Fibonacci numbers:", fibonacci(8));
console.log('Is "Madam" a palindrome?', isPalindrome("Madam"));
console.log('Is "Hello" a palindrome?', isPalindrome("Hello"));
```

## OUTPUT:

The program was run with `node exp3.js` and printed the following output:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS                    + ⌄ bash ... ^ ×

$ node exp3.js
Marks: [ 78, 92, 45, 66, 84 ]
Average: 73.00
Highest / Lowest: { highest: 92, lowest: 45 }

Grade report:
Anil     78  A
Bhavana  92  O
Charan   45  F
Divya    66  B
Eswar    84  A+

Passed students: [ 'Anil', 'Bhavana', 'Divya', 'Eswar' ]
Rank list: 1.Bhavana, 2.Eswar, 3.Anil, 4.Divya, 5.Charan

Factorial of 6: 720
First 8 Fibonacci numbers: [
  0, 1, 1,  2,
  3, 5, 8, 13
]
Is "Madam" a palindrome? true
Is "Hello" a palindrome? false
```

## RESULT:

JavaScript programs using arrays, objects and functions were written and executed successfully.

## VIVA QUESTIONS:

**Q1. What is the difference between `map()` and `forEach()`?**

Ans: `map()` returns a new array of the results; `forEach()` only runs a function and returns nothing.

**Q2. Why is `[...students]` used before sorting?**

Ans: `sort()` changes the array it is called on. Spreading makes a copy so the original order is kept.

**Q3. What is a base case in recursion?**

Ans: The condition that stops the recursion; without it the function would call itself forever.

**Q4. What does `reduce()` need as its starting value?**

Ans: An initial accumulator such as 0; it is passed as the second argument and receives each result in turn.

# EXPERIMENT 4
# DEVELOP DYNAMIC WEBPAGE USING JAVASCRIPT

*[Course Outcome: CO1]*

## AIM:

To develop a dynamic task-board webpage that handles user events and changes the DOM using JavaScript without reloading the page.

## THEORY:

The Document Object Model (DOM) represents the page as a tree of objects. JavaScript can find elements, create new ones, change their text and style, remove them and respond to events such as click, input and keydown. This lets a page update instantly while the user works.

## IMPORTANT CONCEPTS:

- **Selecting elements:** `getElementById` returns one element; `querySelectorAll` returns all that match a CSS selector.
- **Creating and removing nodes:** `createElement`, `append`, `appendChild` and `remove` add and delete list items.
- **Event listeners:** `addEventListener` handles `click`, `keydown` (the Enter key) and `input` (live character counter) without inline attributes.
- **classList:** `classList.toggle("done")` switches a CSS class on and off, used to strike through a completed task and to turn dark mode on and off.
- **Safety and validation:** `textContent` inserts text without parsing HTML, and `trim()` rejects empty input before a task is added.

| Event | Fires when | Used for |
|---|---|---|
| click | A button or item is clicked | Add, delete, complete, theme |
| keydown | A key is pressed | Enter adds the task |
| input | The text box value changes | Live character counter |

## PROCEDURE:

1. Create exp4.html containing a heading, a theme button, a text box, an Add button, an empty list and a statistics line.
2. Add CSS in a style block for the layout, the dark theme and the struck-through completed items.
3. Write the script: updateStats, addTask and the listeners for click, keydown, input and the theme button.
4. Open the page, add three tasks, click one to complete it, delete one and switch to dark mode.
5. Try to add an empty task and confirm that the alert appears.

## SOURCE CODE:

File Name: exp4.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Dynamic Task Board</title>
  <style>
    body { font-family: Arial, sans-serif; max-width: 480px; margin: 30px auto;
        transition: 0.3s; }
    body.dark { background: #1e1e2f; color: #eee; }
    h1 { text-align: center; }
    #row { display: flex; gap: 8px; }
    #taskInput { flex: 1; padding: 8px; }
    button { padding: 8px 12px; cursor: pointer; }
    ul { list-style: none; padding: 0; }
    li { display: flex; justify-content: space-between; padding: 8px; margin-top:
        6px; background: #e8f0fe; border-radius: 6px; color: #222; }
    li.done span { text-decoration: line-through; color: gray; }
    #stats { margin-top: 12px; font-weight: bold; }
    #count { font-size: 12px; color: gray; }
  </style>
</head>
<body>
  <h1>Task Board</h1>
  <button id="themeBtn">Toggle Dark Mode</button>
  <br><br>
  <div id="row">
    <input type="text" id="taskInput" maxlength="40" placeholder="Enter a task">
    <button id="addBtn">Add</button>
  </div>
  <div id="count">0 / 40 characters</div>
  <ul id="taskList"></ul>
  <div id="stats">Total: 0 | Completed: 0</div>

  <script>
```

File Name: exp4.html (continued)

```
    const input = document.getElementById("taskInput");
    const list = document.getElementById("taskList");
    const stats = document.getElementById("stats");

    function updateStats() {
      const total = list.children.length;
      const done = list.querySelectorAll("li.done").length;
      stats.textContent = `Total: ${total} | Completed: ${done}`;
    }

    function addTask() {
      const text = input.value.trim();
      if (text === "") { alert("Please type a task first."); return; }
      const li = document.createElement("li");
      const span = document.createElement("span");
      span.textContent = text;
      span.style.cursor = "pointer";
      span.addEventListener("click", () => { li.classList.toggle("done");
          updateStats(); });
      const del = document.createElement("button");
      del.textContent = "Delete";
      del.addEventListener("click", () => { li.remove(); updateStats(); });
      li.append(span, del);
      list.appendChild(li);
      input.value = "";
      document.getElementById("count").textContent = "0 / 40 characters";
      updateStats();
    }

    document.getElementById("addBtn").addEventListener("click", addTask);
    input.addEventListener("keydown", e => { if (e.key === "Enter") addTask(); });
    input.addEventListener("input", () => {
      document.getElementById("count").textContent = `${input.value.length} / 40
          characters`;
```

File Name: exp4.html (continued)

```
    });
    document.getElementById("themeBtn").addEventListener("click", () => {
      document.body.classList.toggle("dark");
    });
  </script>
</body>
</html>
```

## OUTPUT:

On load the page shows an empty board with Total: 0 | Completed: 0. Each added task appears in the list, clicking its text strikes it through, Delete removes it and the statistics line updates every time. The character counter changes as the user types and the theme button switches between light and dark.

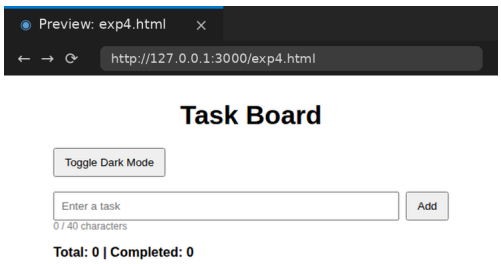



*Figure 4(a): Task board on initial load*

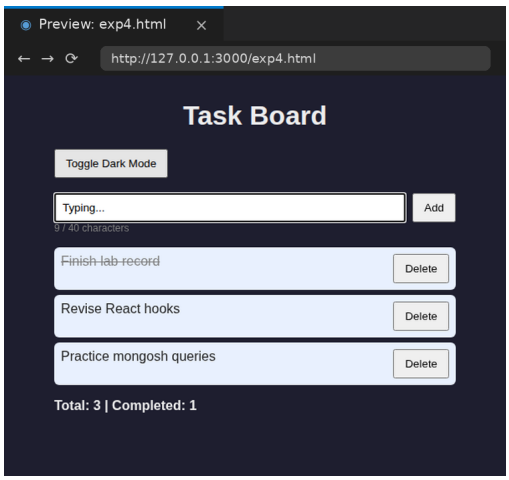



*Figure 4(b): Three tasks added, one completed, dark mode on, text being typed*

## RESULT:

A dynamic webpage responding to user events was developed using JavaScript and the DOM.

## VIVA QUESTIONS:

**Q1. What is the difference between `getElementById` and `querySelector`?**

Ans: `getElementById` finds an element by its id only; `querySelector` accepts any CSS selector and returns the first match.

**Q2. Why use `textContent` instead of `innerHTML` for user input?**

Ans: `textContent` treats the value as plain text, so typed HTML cannot be injected into the page.

**Q3. What does `classList.toggle` do?**

Ans: It adds the class if the element does not have it and removes it if it does.

**Q4. How does the `input` event differ from `change`?**

Ans: `input` fires on every keystroke; `change` fires only after the field loses focus.

# EXPERIMENT 5
# IMPLEMENT MODULES IN NODE.JS

*[Course Outcome: CO2]*

## AIM:

To create a custom module and use the built-in path, fs, os and events modules in Node.js.

## THEORY:

Node.js uses the CommonJS module system in which every file is a module with its own scope. A module shares functions through `module.exports` and another file loads them with `require()`. Node.js also ships built-in modules such as `fs`, `path`, `os` and `events` that need no installation.

## IMPORTANT CONCEPTS:

- **Custom module:** `geometry.js` exports three functions; `app.js` loads them with `require("./geometry")`. The `./` shows that it is a local file.
- **path module:** `path.join`, `path.basename` and `path.extname` build and split file paths in a way that works on every operating system.
- **fs module (asynchronous):** `writeFile`, `appendFile` and `readFile` use callbacks, so Node.js can continue with other work while the disk is busy.
- **events module:** an `EventEmitter` registers a listener with `on()` and triggers it with `emit()`; this is the pattern behind many Node.js features.
- **Order of output:** because file operations are asynchronous, the file content is printed after the lines that follow them in the code.

| Module | Use |
|---|---|
| fs | Read, write, append and delete files |
| path | Join and split file paths |
| os | Information about the operating system and hardware |
| events | Create and handle custom events |

## PROCEDURE:

1. Create geometry.js with the three functions and export them with module.exports.
2. Create app.js, import geometry.js and the four built-in modules.
3. Call the geometry functions, build a path, write, append and read notes.txt, and emit an event.
4. Run node app.js.
5. Open the folder to confirm that notes.txt was created and observe that the file content appears last.

## SETUP AND RUN:

Commands

```
node app.js
```

## SOURCE CODE:

File Name: geometry.js

```
// Custom module: geometry helpers
const PI = Math.PI;

function circleArea(r) {
  return +(PI * r * r).toFixed(2);
}

function rectangleArea(l, b) {
  return l * b;
}

function celsiusToFahrenheit(c) {
  return +(c * 9 / 5 + 32).toFixed(1);
}

module.exports = { circleArea, rectangleArea, celsiusToFahrenheit };
```

File Name: app.js

```
const geometry = require("./geometry");   // custom module
const fs = require("fs");                 // built-in modules
const os = require("os");
const path = require("path");
const EventEmitter = require("events");

console.log("Circle area (r = 7):", geometry.circleArea(7));
console.log("Rectangle area (5 x 8):", geometry.rectangleArea(5, 8));
console.log("37 C in Fahrenheit:", geometry.celsiusToFahrenheit(37));

// path module
const file = path.join(__dirname, "notes.txt");
console.log("Base name:", path.basename(file), "| Extension:", path.extname(file));

// fs module: write, append and read asynchronously
fs.writeFile(file, "Line 1: Node.js modules\n", err => {
  if (err) throw err;
  fs.appendFile(file, "Line 2: written with the fs module\n", err => {
    if (err) throw err;
    fs.readFile(file, "utf8", (err, data) => {
      if (err) throw err;
      console.log("File content:\n" + data.trim());
      console.log("Lines in file:", data.trim().split("\n").length);
    });
  });
});

// os module
console.log("CPU cores available:", os.cpus().length > 0 ? "yes" : "no");
console.log("Total memory > 0 MB:", os.totalmem() / 1048576 > 0);

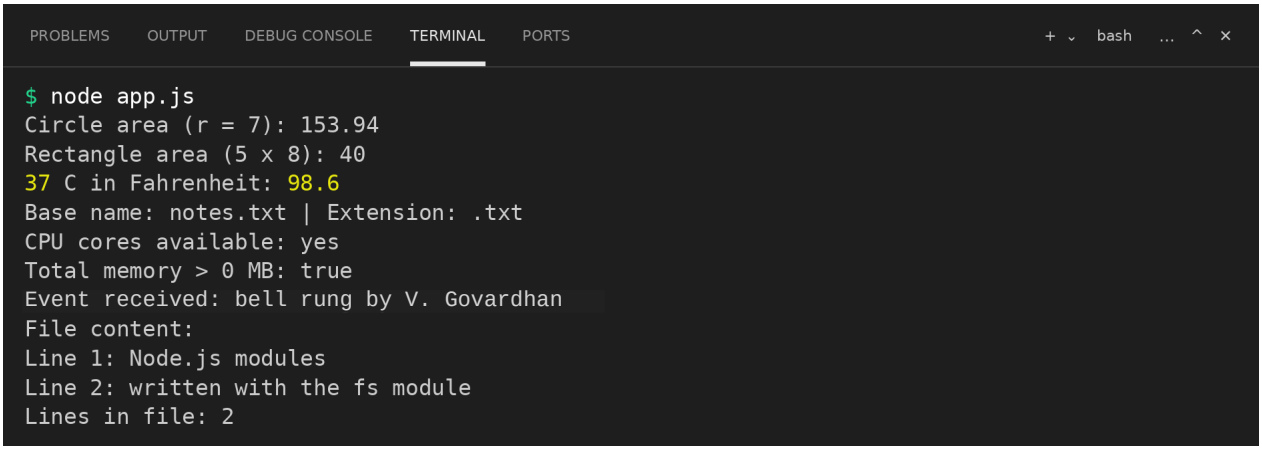
// events module
const bell = new EventEmitter();
bell.on("ring", who => console.log("Event received: bell rung by", who));
```

File Name: app.js (continued)

```
bell.emit("ring", "V. Govardhan");
```

## OUTPUT:

The file content is printed last because readFile is asynchronous.




## RESULT:

A custom module and built-in modules were created, imported and used successfully in Node.js.

## VIVA QUESTIONS:

**Q1. What does `require()` return?**

Ans: The value that the loaded module assigned to `module.exports`.

**Q2. Why does the file content print after the other lines?**

Ans: `readFile` is asynchronous; its callback runs only after the synchronous code has finished.

**Q3. Why use `path.join` instead of joining strings with `/`?**

Ans: `path.join` uses the correct separator for the operating system, so the code works on Windows, Linux and macOS.

**Q4. What is an EventEmitter?**

Ans: An object that lets code register listeners for named events with `on()` and trigger them with `emit()`.

# EXPERIMENT 6
# DEVELOP MINI APPLICATION USING EXPRESS AND NODE.JS

*[Course Outcome: CO2]*

## AIM:

To develop a library REST API with Express and Node.js that uses routing, middleware, query strings, route parameters, input validation and proper status codes.

## THEORY:

Express is a minimal web framework for Node.js. Routes connect an HTTP method and a URL to a handler function. Middleware functions run before the handlers and can log requests, parse JSON or stop invalid requests. A REST API treats data as resources, such as `/books`, and uses the HTTP method to describe the action.

## IMPORTANT CONCEPTS:

- **Routing:** `app.get`, `app.post`, `app.put` and `app.delete` handle the four operations on the books resource.
- **Route parameters and query strings:** `/books/:id` is read from `req.params.id`; `/books?available=true` is read from `req.query.available`.
- **Middleware:** `express.json()` parses JSON bodies and a custom logger prints the method, URL and time of every request before calling `next()`.
- **Validation:** POST returns status 400 when the title or author is missing, instead of storing bad data.
- **Fallback route:** the last `app.use` returns a 404 JSON message for any unknown URL.
- **Limitation:** books are kept in an array, so they disappear when the server restarts. Experiment 9 adds a database.

| Status | Meaning | Returned by |
|---|---|---|
| 200 | OK | GET, PUT, DELETE |
| 201 | Created | POST on success |
| 400 | Bad Request | POST with missing fields |
| 404 | Not Found | Unknown id or route |

## PROCEDURE:

1. Create the project folder, run npm init -y and install express.
2. Write server.js with the logger, the five routes and the fallback handler.
3. Start the server with node server.js and open http://localhost:3000/books in a browser.
4. Send POST, PUT and DELETE requests with curl or Postman.
5. Send a POST without an author and a GET for a missing id to confirm the 400 and 404 responses.

## SETUP AND RUN:

Commands

```
mkdir library-api && cd library-api
npm init -y
npm install express
node server.js
```

## SOURCE CODE:

File Name: server.js

```
const express = require("express");
const app = express();
const PORT = 3000;

app.use(express.json());

// Logging middleware: prints method, URL and time of every request
app.use((req, res, next) => {
  console.log(`${new Date().toLocaleTimeString()}  ${req.method} ${req.url}`);
  next();
});

let books = [
  { id: 1, title: "Clean Code", author: "Robert C. Martin", available: true },
  { id: 2, title: "Python Crash Course", author: "Eric Matthes", available: true },
  { id: 3, title: "Deep Learning", author: "Ian Goodfellow", available: false }
];
let nextId = 4;

app.get("/", (req, res) => {
  res.send("<h2>ANITS Library API</h2><p>Try <a href='/books'>/books</a></p>");
});

// READ: all books, with optional ?available=true filter
app.get("/books", (req, res) => {
  if (req.query.available !== undefined) {
    const wanted = req.query.available === "true";
    return res.json(books.filter(b => b.available === wanted));
  }
  res.json(books);
});

// READ: one book by id
app.get("/books/:id", (req, res) => {
```

File Name: server.js (continued)

```
  const book = books.find(b => b.id === Number(req.params.id));
  if (!book) return res.status(404).json({ error: "Book not found" });
  res.json(book);
});

// CREATE: add a book (title and author are mandatory)
app.post("/books", (req, res) => {
  const { title, author } = req.body;
  if (!title || !author) {
    return res.status(400).json({ error: "title and author are required" });
  }
  const book = { id: nextId++, title, author, available: true };
  books.push(book);
  res.status(201).json(book);
});

// UPDATE: change availability or details
app.put("/books/:id", (req, res) => {
  const book = books.find(b => b.id === Number(req.params.id));
  if (!book) return res.status(404).json({ error: "Book not found" });
  Object.assign(book, req.body);
  res.json(book);
});

// DELETE
app.delete("/books/:id", (req, res) => {
  const before = books.length;
  books = books.filter(b => b.id !== Number(req.params.id));
  if (books.length === before) return res.status(404).json({ error: "Book not found"
      });
  res.json({ message: "Book removed" });
});

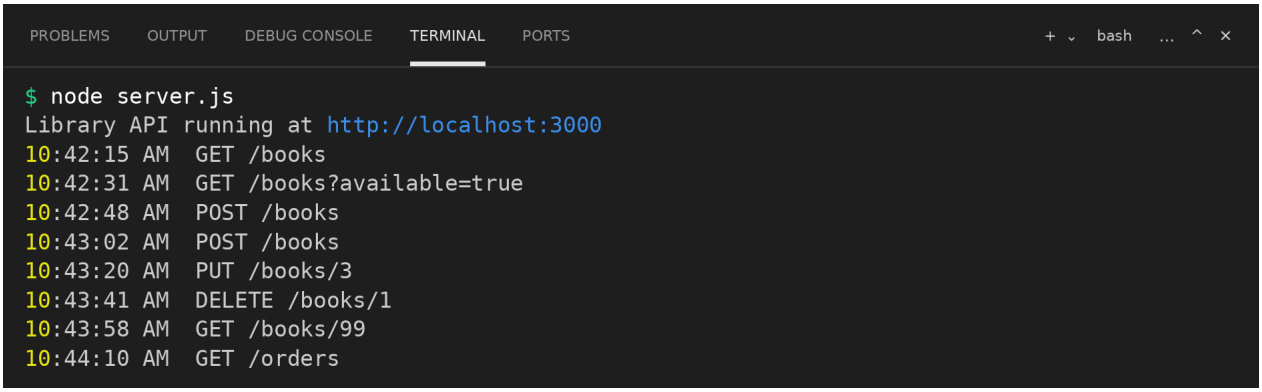
// Fallback for unknown routes
```

File Name: server.js (continued)

```
app.use((req, res) => res.status(404).json({ error: "Route not found" }));

app.listen(PORT, () => console.log(`Library API running at
    http://localhost:${PORT}`));
```

## OUTPUT:




Requests sent with curl while the server was running, and the responses received:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS                    + ⌄ bash ... ^ ×

$ curl -s -w "\nHTTP %{http_code}\n" http://localhost:3000/books
[{"id":1,"title":"Clean Code","author":"Robert C.
    Martin","available":true},{"id":2,"title":"Python Crash Course","author":"Eric
    Matthes","available":true},{"id":3,"title":"Deep Learning","author":"Ian
    Goodfellow","available":false}]
HTTP 200

$ curl -s -w "\nHTTP %{http_code}\n" "http://localhost:3000/books?available=true"
[{"id":1,"title":"Clean Code","author":"Robert C.
    Martin","available":true},{"id":2,"title":"Python Crash Course","author":"Eric
    Matthes","available":true}]
HTTP 200

$ curl -s -w "\nHTTP %{http_code}\n" -X POST http://localhost:3000/books -H
    "Content-Type: application/json" -d '{"title":"AI"}'
{"error":"title and author are required"}
HTTP 400

$ curl -s -w "\nHTTP %{http_code}\n" -X POST http://localhost:3000/books -H
    "Content-Type: application/json" -d '{"title":"Hands-On ML","author":"Aurelien
    Geron"}'
{"id":4,"title":"Hands-On ML","author":"Aurelien Geron","available":true}
HTTP 201

$ curl -s -w "\nHTTP %{http_code}\n" -X PUT http://localhost:3000/books/3 -H
    "Content-Type: application/json" -d '{"available":true}'
{"id":3,"title":"Deep Learning","author":"Ian Goodfellow","available":true}
HTTP 200

$ curl -s -w "\nHTTP %{http_code}\n" -X DELETE http://localhost:3000/books/1
{"message":"Book removed"}
HTTP 200

$ curl -s -w "\nHTTP %{http_code}\n" http://localhost:3000/books/99
{"error":"Book not found"}
HTTP 404

$ curl -s -w "\nHTTP %{http_code}\n" http://localhost:3000/orders
{"error":"Route not found"}
HTTP 404
```

## RESULT:

A library REST API with routing, middleware and validation was developed using Express and Node.js.

## VIVA QUESTIONS:

**Q1. What is the role of `next()` in middleware?**

Ans: It passes control to the next middleware or route handler; without it the request would hang.

**Q2. Why return 400 for a missing title?**

Ans: 400 tells the client that the request itself was invalid, so the fault is on the client side and nothing was stored.

**Q3. What is the difference between `req.params` and `req.query`?**

Ans: `req.params` holds values from the path (like :id); `req.query` holds values after the ? in the URL.

**Q4. Why is the 404 handler placed last?**

Ans: Express checks routes in order; the catch-all must come after every real route or it would answer for them too.

# EXPERIMENT 7
# CREATE FUNCTIONAL AND CLASS COMPONENTS USING REACT

*[Course Outcome: CO3]*

## AIM:

To create functional and class components in React that use props, state, event handling, list rendering and lifecycle cleanup.

## THEORY:

React builds a user interface from small reusable components. A functional component is a function that returns JSX and keeps state with hooks such as `useState`. A class component extends `React.Component`, keeps state in `this.state`, changes it with `setState()` and can run code in lifecycle methods. Props pass read-only data from a parent to a child.

## IMPORTANT CONCEPTS:

- **Props:** `ProfileCard` receives `name`, `branch` and `skills` from `App`; the child only reads them.
- **List rendering:** `skills.map(...)` creates one `<li>` per skill and `key={s}` lets React track each item.
- **State with hooks:** `LikeButton` uses two `useState` values and a toggle function that adds or removes one like.
- **Class state and lifecycle:** `Stopwatch` stores `seconds` and `running`, starts a timer with `setInterval` and clears it in `componentWillUnmount` to avoid a memory leak.
- **Arrow-function class properties:** `start = () => {...}` keeps `this` bound to the component when used as an event handler.
- **Vite:** `npm run dev` starts a fast development server at http://localhost:5173.

| Feature | Functional | Class |
|---|---|---|
| State | useState hook | this.state and setState() |
| Cleanup | useEffect return function | componentWillUnmount() |
| Props | Function argument | this.props |
| Modern usage | Preferred | Legacy, still supported |

## PROCEDURE:

1. Create the project with npm create vite@latest and choose React.
2. Create src/Components.jsx with ProfileCard, LikeButton and Stopwatch.
3. Replace src/App.jsx so that it renders the three components, and keep the generated src/main.jsx.
4. Run npm run dev and open http://localhost:5173.
5. Click Like twice, press Start on the stopwatch, wait a few seconds, then Stop and Reset.

## SETUP AND RUN:

Commands

```
npm create vite@latest react-components -- --template react
cd react-components
npm install
npm run dev
```

## SOURCE CODE:

File Name: src/Components.jsx

```
import { useState, Component } from "react";

// Functional component with props
export function ProfileCard({ name, branch, skills }) {
  return (
    <div style={{ border: "1px solid #999", borderRadius: 8, padding: 12, width: 260,
        margin: "10px auto" }}>
      <h3>{name}</h3>
      <p>Branch: {branch}</p>
      <ul style={{ textAlign: "left" }}>
        {skills.map(s => <li key={s}>{s}</li>)}
      </ul>
    </div>
  );
}

// Functional component with state (useState)
export function LikeButton() {
  const [likes, setLikes] = useState(0);
  const [liked, setLiked] = useState(false);

  const toggle = () => {
    setLikes(liked ? likes - 1 : likes + 1);
    setLiked(!liked);
  };

  return (
    <div>
      <h3>Like Button (functional)</h3>
      <button onClick={toggle}>{liked ? "Unlike" : "Like"}</button>
      <p>Total likes: {likes}</p>
    </div>
  );
}
```

File Name: src/Components.jsx (continued)

```
// Class component with state and lifecycle methods
export class Stopwatch extends Component {
  constructor(props) {
    super(props);
    this.state = { seconds: 0, running: false };
    this.timer = null;
  }

  start = () => {
    if (this.state.running) return;
    this.setState({ running: true });
    this.timer = setInterval(() => this.setState({ seconds: this.state.seconds + 1
        }), 1000);
  };

  stop = () => {
    clearInterval(this.timer);
    this.setState({ running: false });
  };

  reset = () => {
    this.stop();
    this.setState({ seconds: 0 });
  };

  componentWillUnmount() {
    clearInterval(this.timer);   // cleanup when the component is removed
  }

  render() {
    return (
      <div>
        <h3>Stopwatch (class)</h3>
```

File Name: src/Components.jsx (continued)

```
        <p style={{ fontSize: 28 }}>{this.state.seconds} s</p>
        <button onClick={this.start}>Start</button>{" "}
        <button onClick={this.stop}>Stop</button>{" "}
        <button onClick={this.reset}>Reset</button>
      </div>
    );
  }
}
```

File Name: src/App.jsx

```
import { ProfileCard, LikeButton, Stopwatch } from "./Components";

function App() {
  return (
    <div style={{ textAlign: "center", fontFamily: "Arial" }}>
      <h1>React Components Experiment</h1>
      <ProfileCard
        name="V. Govardhan"
        branch="CSE (AI&ML)"
        skills={["Python", "JavaScript", "React"]}
      />
      <LikeButton />
      <hr />
      <Stopwatch />
    </div>
  );
}

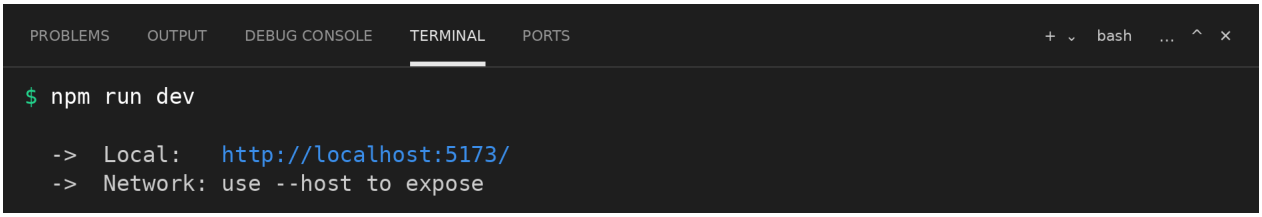
export default App;
```

File Name: src/main.jsx

```
import { createRoot } from "react-dom/client";
import App from "./App.jsx";

createRoot(document.getElementById("root")).render(<App />);
```

## OUTPUT:




The page at http://localhost:5173 responds to each action as shown in the table.

| Action performed | What the page displays |
|---|---|
| Page loads | Heading "React Components Experiment"; profile card for V. Govardhan, Branch: CSE (AI&ML), skills Python, JavaScript, React; Like button with Total likes: 0; Stopwatch showing 0 s |
| Click Like | Button label changes to Unlike; Total likes: 1 |
| Click Unlike | Button label returns to Like; Total likes: 0 |
| Click Start and wait 5 seconds | Stopwatch counts 1 s, 2 s, 3 s, 4 s, 5 s |
| Click Stop | The counter stops at the current value (5 s) |
| Click Start again | Counting continues from 5 s |
| Click Reset | Counter returns to 0 s and stops |

## RESULT:

Functional and class components were created in React using props, state and lifecycle cleanup.

## VIVA QUESTIONS:

**Q1. Why is a `key` needed in `skills.map()`?**

Ans: It gives each item a stable identity so React updates only the items that changed.

**Q2. Why is `clearInterval` called in `componentWillUnmount`?**

Ans: Otherwise the timer would keep running after the component is removed, wasting memory and updating nothing.

**Q3. Can a child component change its props?**

Ans: No. Props are read-only; the child must ask the parent to change the data, usually through a function passed as a prop.

**Q4. What happens when `setLikes` is called?**

Ans: React stores the new value and re-renders the component so the screen shows the updated count.

# EXPERIMENT 8
# IMPLEMENT HTTP METHODS USING JSON SERVER

*[Course Outcome: CO3]*

## AIM:

To perform GET, POST, PUT, PATCH and DELETE operations, with filtering and sorting, on a fake REST API created with JSON Server.

## THEORY:

JSON Server turns a JSON file into a working REST API without writing any server code. Every top-level array in `db.json` becomes a resource with routes such as `/books` and `/books/:id`. The HTTP methods map to CRUD: GET reads, POST creates, PUT replaces, PATCH changes only some fields and DELETE removes.

## IMPORTANT CONCEPTS:

- **Resources:** `db.json` has two arrays, `books` and `members`, so JSON Server creates `/books` and `/members`.
- **PUT and PATCH:** PUT must send the whole record and replaces it; PATCH sends only the changed field, for example `{"stock": 10}`.
- **Filtering and sorting:** `?author=Eric%20Matthes` filters by a field; `?_sort=price&_order=desc` sorts by price from high to low.
- **Auto id:** a POST without an id receives the next id automatically.
- **Content-Type header:** `application/json` is required so that the body is parsed as JSON.
- **Version:** this record uses JSON Server 0.17.4 because newer versions changed the command line options.

| Method | Operation | Idempotent? | Success status |
|---|---|---|---|
| GET | Read | Yes | 200 |
| POST | Create | No | 201 |
| PUT | Replace the whole record | Yes | 200 |
| PATCH | Update some fields | Usually | 200 |
| DELETE | Remove | Yes | 200 |

## PROCEDURE:

1. Create a folder and install JSON Server 0.17.4 with npm.
2. Create db.json with the books and members arrays.
3. Start the server on port 4000 with the --watch option.
4. In a second terminal run bash requests.sh to send the nine requests in order.
5. Open db.json afterwards and confirm that the changes were saved to the file.

## SETUP AND RUN:

Commands

```
mkdir json-server-demo && cd json-server-demo
npm init -y
npm install json-server@0.17.4
npx json-server --watch db.json --port 4000
# second terminal
bash requests.sh
```

## SOURCE CODE:

File Name: db.json

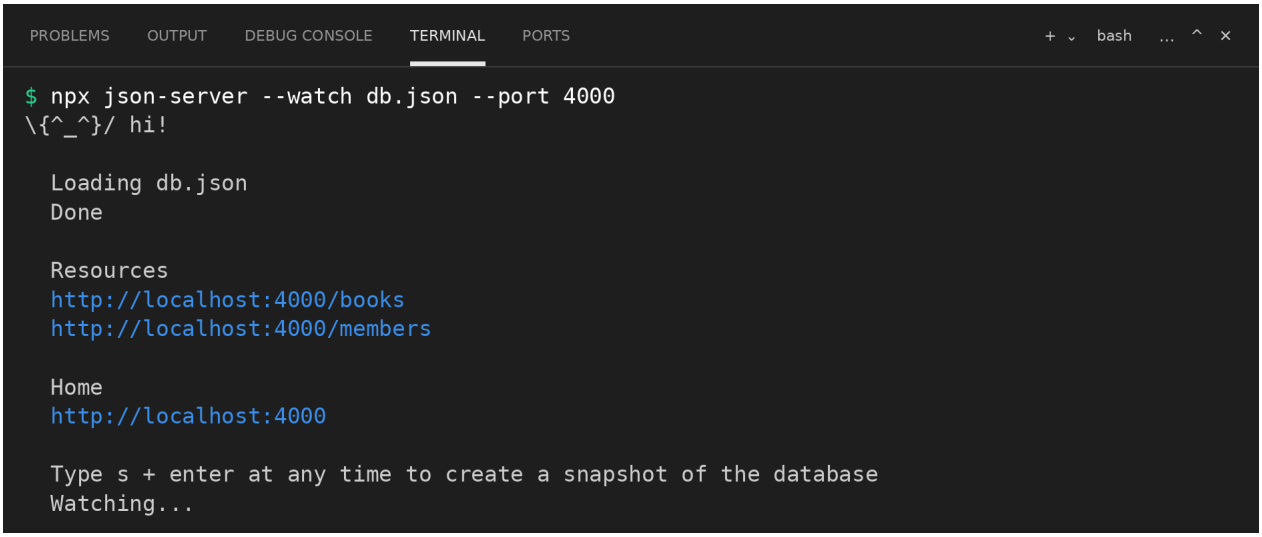
```
{
  "books": [
    { "id": 1, "title": "Clean Code", "author": "Robert C. Martin", "price": 450,
        "stock": 6 },
    { "id": 2, "title": "Python Crash Course", "author": "Eric Matthes", "price":
        600, "stock": 3 }
  ],
  "members": [
    { "id": 1, "name": "Ravi", "year": 2 }
  ]
}
```

File Name: requests.sh

```
#!/bin/bash
# Experiment 8 - HTTP methods against JSON Server (http://localhost:4000)
BASE=http://localhost:4000/books
FMT='\nHTTP %{http_code}\n'
H='Content-Type: application/json'

echo '$ GET all books';            curl -s -w "$FMT" $BASE
echo '$ GET one book';             curl -s -w "$FMT" $BASE/2
echo '$ GET with filter';          curl -s -w "$FMT" "$BASE?author=Eric%20Matthes"
echo '$ GET sorted by price';      curl -s -w "$FMT" "$BASE?_sort=price&_order=desc"
echo '$ POST a new book';          curl -s -w "$FMT" -X POST $BASE -H "$H" \
   -d '{"title":"Hands-On ML","author":"Aurelien Geron","price":900,"stock":2}'
echo '$ PUT (replace) book 1';     curl -s -w "$FMT" -X PUT $BASE/1 -H "$H" \
   -d '{"title":"Clean Code 2nd Ed","author":"Robert C.
       Martin","price":500,"stock":8}'
echo '$ PATCH (stock only) book 2'; curl -s -w "$FMT" -X PATCH $BASE/2 -H "$H" -d
    '{"stock":10}'
echo '$ DELETE book 1';            curl -s -w "$FMT" -X DELETE $BASE/1
echo '$ GET all books (final)';    curl -s -w "$FMT" $BASE
```

**OUTPUT:**




Responses printed by requests.sh in the second terminal. POST received the automatic id 3, PUT replaced book 1, PATCH changed only the stock of book 2 and DELETE returned an empty object.


PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
bash
$ bash requests.sh
$ GET all books
[
{
"id": 1,
"title": "Clean Code",
"author": "Robert C. Martin",
"price": 450,
"stock": 6
},
{
"id": 2,
"title": "Python Crash Course",
"author": "Eric Matthes",
"price": 600,
"stock": 3
}
]
HTTP 200
$ GET one book
{
"id": 2,
"title": "Python Crash Course",
"author": "Eric Matthes",
"price": 600,
"stock": 3
}
HTTP 200
$ GET with filter
[
{
"id": 2,
"title": "Python Crash Course",
"author": "Eric Matthes",
"price": 600,
"stock": 3
}
]
HTTP 200
$ GET sorted by price
[
{
"id": 2,
"title": "Python Crash Course",
"author": "Eric Matthes",
"price": 600,
"stock": 3
},
{
"id": 1,

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS                    + ⌄ bash ... ^ ×

    "title": "Clean Code",
    "author": "Robert C. Martin",
    "price": 450,
    "stock": 6
  }
]
HTTP 200
$ POST a new book
{
  "title": "Hands-On ML",
  "author": "Aurelien Geron",
  "price": 900,
  "stock": 2,
  "id": 3
}
HTTP 201
$ PUT (replace) book 1
{
  "title": "Clean Code 2nd Ed",
  "author": "Robert C. Martin",
  "price": 500,
  "stock": 8,
  "id": 1
}
HTTP 200
$ PATCH (stock only) book 2
{
  "id": 2,
  "title": "Python Crash Course",
  "author": "Eric Matthes",
  "price": 600,
  "stock": 10
}
HTTP 200
$ DELETE book 1
{}
HTTP 200
$ GET all books (final)
[
  {
    "id": 2,
    "title": "Python Crash Course",
    "author": "Eric Matthes",
    "price": 600,
    "stock": 10
  },
  {
    "title": "Hands-On ML",
    "author": "Aurelien Geron",
    "price": 900,
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS                    + ⌄ bash ... ^ ×

    "stock": 2,
    "id": 3
  }
]
HTTP 200
```

## RESULT:

All five HTTP methods, with filtering and sorting, were implemented against a JSON Server REST API.

## VIVA QUESTIONS:

**Q1. Which JSON Server route does the `books` array create?**

Ans: `/books` for the whole collection and `/books/:id` for a single record.

**Q2. What happens to the other fields in a PATCH request?**

Ans: They stay unchanged; only the fields sent in the body are updated.

**Q3. Why is DELETE called idempotent?**

Ans: Deleting the same record again leaves the data in the same final state as deleting it once.

**Q4. How does `_sort=price&_order=desc` behave?**

Ans: It returns the records ordered by price from the highest to the lowest.

# EXPERIMENT 9
# DEVELOP CRUD OPERATIONS USING MONGODB (MONGOOSE AND MONGOSH)

*[Course Outcome: CO4]*

## AIM:

To perform Create, Read, Update and Delete operations and an aggregation on a students collection, first with the MongoDB shell (mongosh) and then from Node.js using Mongoose with validation.

## THEORY:

MongoDB is a NoSQL document database. Data is stored as BSON documents inside collections, and a database holds many collections. Mongoose is an object data modelling library for Node.js. It adds schemas, data types, validation rules and a simple API on top of MongoDB.

## IMPORTANT CONCEPTS:

- **Hierarchy:** database `universityDB` contains the collection `students`, which contains documents with fields such as name, branch, cgpa and skills.
- **Query operators:** `$gte` selects CGPA of 8.5 or more; matching a value inside an array (`{ skills: "Python" }`) needs no special operator.
- **Projection, sort and limit:** `{ name: 1, cgpa: 1, _id: 0 }` chooses the fields returned, `sort({ cgpa: -1 })` orders highest first and `limit(2)` keeps the top two.
- **Update operators:** `$set` changes a value, `$inc` adds to a number and `$push` appends to an array.
- **Aggregation:** `$group` with `$avg` and `$sum` finds the average CGPA and the number of students in every branch.
- **Mongoose validation:** the schema uses `required`, `enum`, `min` and `max`, so a student with no name and CGPA 12 is rejected.

| Operator | Meaning | Example |
|---|---|---|
| $gte / $lt | Greater or equal / less than | { cgpa: { $gte: 8.5 } } |
| $set | Set a field | { $set: { cgpa: 7.9 } } |
| $inc | Add to a number | { $inc: { cgpa: 0.1 } } |
| $push | Append to an array | { $push: { skills: "MongoDB" } } |
| $group | Group and summarise | { $group: { _id: "$branch" } } |

## PROCEDURE:

1. Start the MongoDB service and open a terminal.
2. Run mongosh --file mongosh_commands.js, or paste the commands one by one, and inspect the results.
3. Create a Node.js project and install mongoose.
4. Write crud.js with the schema and the five steps: insert, read, update, delete and validate.

5. Run node crud.js and compare the output with the expected output.

## SETUP AND RUN:

Commands

```
# Part A - mongosh
mongosh
use universityDB
show collections

# Part B - Mongoose
mkdir mongo-crud && cd mongo-crud
npm init -y
npm install mongoose
node crud.js
```

## SOURCE CODE:

File Name: mongosh_commands.js

```
// Run inside mongosh: mongosh --file mongosh_commands.js
use("universityDB");

// CREATE
db.students.insertOne({ name: "Ravi", branch: "CSE", cgpa: 8.4, skills: ["Python",
    "SQL"] });
db.students.insertMany([
  { name: "Sneha", branch: "AIML", cgpa: 9.1, skills: ["ML", "Python"] },
  { name: "Kiran", branch: "ECE", cgpa: 7.2, skills: ["VLSI"] },
  { name: "Divya", branch: "AIML", cgpa: 8.8, skills: ["React", "Node"] }
]);

// READ
db.students.find();                                              // all
db.students.find({ branch: "AIML" });                            // equality filter
db.students.find({ cgpa: { $gte: 8.5 } }, { name: 1, cgpa: 1, _id: 0 });   //
    projection
db.students.find({ skills: "Python" }).sort({ cgpa: -1 });       // array match + sort
db.students.find().sort({ cgpa: -1 }).limit(2);                  // top two
db.students.countDocuments({ branch: "AIML" });

// UPDATE
db.students.updateOne({ name: "Kiran" }, { $set: { cgpa: 7.9 } });
db.students.updateMany({ branch: "AIML" }, { $inc: { cgpa: 0.1 } });
db.students.updateOne({ name: "Ravi" }, { $push: { skills: "MongoDB" } });

// DELETE
db.students.deleteOne({ name: "Kiran" });
db.students.deleteMany({ cgpa: { $lt: 5 } });

// AGGREGATION: average CGPA per branch
db.students.aggregate([
  { $group: { _id: "$branch", avgCgpa: { $avg: "$cgpa" }, count: { $sum: 1 } } }
]);
```

File Name: crud.js

```
const mongoose = require("mongoose");

const studentSchema = new mongoose.Schema({
  name:   { type: String, required: true, trim: true },
  branch: { type: String, enum: ["CSE", "AIML", "ECE", "IT"], default: "CSE" },
  cgpa:   { type: Number, min: 0, max: 10 },
  skills: [String]
});
const Student = mongoose.model("Student", studentSchema);

async function main() {
  await mongoose.connect("mongodb://127.0.0.1:27017/universityDB");
  console.log("Connected to MongoDB");

  await Student.deleteMany({});                      // start clean

  // CREATE
  await Student.insertMany([
    { name: "Ravi",  branch: "CSE",  cgpa: 8.4, skills: ["Python", "SQL"] },
    { name: "Sneha", branch: "AIML", cgpa: 9.1, skills: ["ML", "Python"] },
    { name: "Kiran", branch: "ECE",  cgpa: 7.2, skills: ["VLSI"] }
  ]);
  console.log("Inserted 3 students");

  // READ
  const toppers = await Student.find({ cgpa: { $gte: 8 } }).sort({ cgpa: -1 });
  console.log("Students with CGPA >= 8:", toppers.map(s => `${s.name} (${s.cgpa})`));

  // UPDATE
  const updated = await Student.findOneAndUpdate(
    { name: "Kiran" }, { cgpa: 7.9 }, { new: true });
  console.log("Updated Kiran's CGPA to", updated.cgpa);

  // DELETE
```

File Name: crud.js (continued)

```
  const removed = await Student.deleteOne({ name: "Ravi" });
  console.log("Deleted documents:", removed.deletedCount);

  // VALIDATION
  try {
    await Student.create({ branch: "IT", cgpa: 12 });
  } catch (err) {
    console.log("Validation failed:", Object.keys(err.errors).join(", "));
  }

  console.log("Remaining students:", await Student.countDocuments());
  await mongoose.disconnect();
}

main().catch(err => console.error("Error:", err.message));
```

## OUTPUT:

Part A - commands run in mongosh and the results shown by the shell:

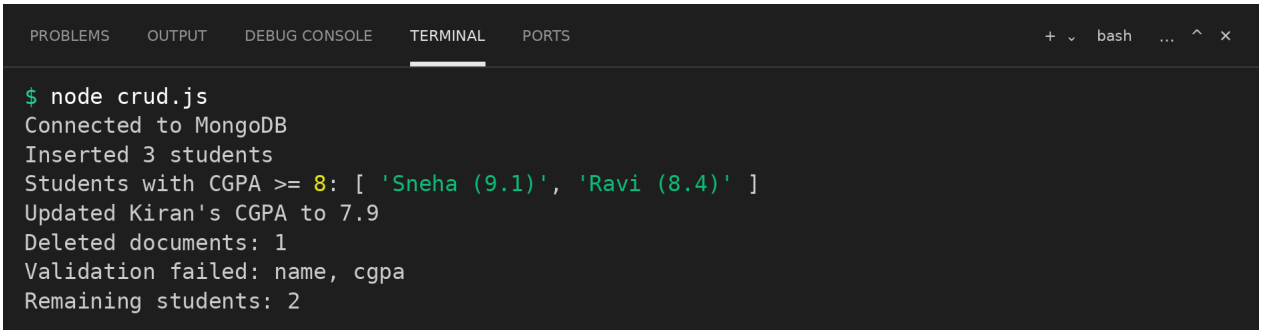
```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS                    + ⌄ mongosh ... ^ ×

test> use("universityDB")
switched to db universityDB
universityDB> db.students.insertOne({ name: "Ravi", branch: "CSE", cgpa: 8.4, skills:
    ["Python", "SQL"] })
{ acknowledged: true, insertedId: ObjectId('...') }
universityDB> db.students.insertMany([ ...Sneha, Kiran, Divya... ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('...'),
    '1': ObjectId('...'),
    '2': ObjectId('...')
  }
}
universityDB> db.students.find({ branch: "AIML" })
[
  {
    _id: ObjectId('...'),
    name: 'Sneha',
    branch: 'AIML',
    cgpa: 9.1,
    skills: [ 'ML', 'Python' ]
  },
  {
    _id: ObjectId('...'),
    name: 'Divya',
    branch: 'AIML',
    cgpa: 8.8,
    skills: [ 'React', 'Node' ]
  }
]
universityDB> db.students.find({ cgpa: { $gte: 8.5 } }, { name: 1, cgpa: 1, _id: 0 })
[ { name: 'Sneha', cgpa: 9.1 }, { name: 'Divya', cgpa: 8.8 } ]
universityDB> db.students.countDocuments({ branch: "AIML" })
2
universityDB> db.students.updateOne({ name: "Kiran" }, { $set: { cgpa: 7.9 } })
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
universityDB> db.students.updateMany({ branch: "AIML" }, { $inc: { cgpa: 0.1 } })
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 2,
  modifiedCount: 2,
  upsertedCount: 0
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS                    + ⌄ mongosh ... ^ ×

}
universityDB> db.students.deleteOne({ name: "Kiran" })
{ acknowledged: true, deletedCount: 1 }
universityDB> db.students.aggregate([{ $group: { _id: "$branch", avgCgpa: { $avg:
    "$cgpa" }, count: { $sum: 1 } } }])
[
  { _id: 'AIML', avgCgpa: 9.05, count: 2 },
  { _id: 'CSE', avgCgpa: 8.4, count: 1 }
]
```

Part B - output of the Mongoose program:




## RESULT:

CRUD operations and an aggregation were written for MongoDB using mongosh and Mongoose with schema validation.

## VIVA QUESTIONS:

**Q1. What does a projection do?**

Ans: It chooses which fields appear in the result, for example `{ name: 1, _id: 0 }` returns only the name.

**Q2. What is the difference between `$set` and `$inc`?**

Ans: `$set` replaces a value; `$inc` adds a number to the existing value.

**Q3. Why does the Mongoose validation error list two fields?**

Ans: The document has no name (required) and a CGPA of 12 (above the maximum of 10), so both rules fail.

**Q4. What is aggregation used for?**

Ans: To process many documents in stages, for example grouping them and calculating averages or counts.

# EXPERIMENT 10
# DEVELOP MAIN PROJECT USING EXPRESS, REACT AND MONGODB

*[Course Outcome: CO4]*

## AIM:

To develop a full-stack Library Book Manager using Express for the API, React for the interface and MongoDB with Mongoose for storage, supporting add, search, edit, issue/return and delete.

## THEORY:

A MERN application has three layers. MongoDB stores the data, Express with Node.js exposes it through a REST API and React shows it to the user and sends the user's actions back to the API with `fetch`. Because the React page (port 5173) and the API (port 5000) are different origins, the server enables CORS.

## IMPORTANT CONCEPTS:

- **Schema:** a Book has a required title and author, a price (minimum 0) and an `available` flag; `timestamps: true` adds createdAt and updatedAt automatically.
- **Search:** the API accepts `?search=` and filters titles with a case-insensitive regular expression; React re-fetches whenever the search text changes through `useEffect(..., [search])`.
- **Controlled inputs:** every field gets its value from state and updates it in `onChange`; one form is used for both Add and Update, chosen by `editId`.
- **Issue and return:** a PUT with `{ available: !b.available }` toggles the status of one book.
- **Error handling:** the server returns 400 for validation errors and 404 for missing books; the client shows a message if the server cannot be reached.
- **CRUD mapping:** Create = POST, Read = GET, Update = PUT, Delete = DELETE.

| Layer | Technology | Port |
|---|---|---|
| Database | MongoDB with Mongoose | 27017 |
| Server | Node.js and Express | 5000 |
| Client | React with Vite | 5173 |

## PROCEDURE:

1. Start MongoDB, then run node server.js inside the server folder and wait for MongoDB connected.
2. Create the Vite React app in the client folder, replace src/App.jsx and src/main.jsx with the code below and run npm run dev.
3. Open http://localhost:5173 and add three books.
4. Type part of a title in the search box and confirm that the table is filtered.
5. Click Issue, Edit and Delete on different rows and check the changes with db.books.find() in mongosh.

## SETUP AND RUN:

Commands

```
mkdir library-manager && cd library-manager
mkdir server && cd server
npm init -y
npm install express mongoose cors
node server.js          # terminal 1
cd ..
npm create vite@latest client -- --template react
cd client
npm install
npm run dev             # terminal 2
```

## SOURCE CODE:

File Name: server/server.js

```
const express = require("express");
const mongoose = require("mongoose");
const cors = require("cors");

const app = express();
app.use(cors());
app.use(express.json());

mongoose.connect("mongodb://127.0.0.1:27017/libraryDB")
  .then(() => console.log("MongoDB connected"))
  .catch(err => console.error("Connection error:", err.message));

const Book = mongoose.model("Book", new mongoose.Schema({
  title:     { type: String, required: true, trim: true },
  author:    { type: String, required: true, trim: true },
  price:     { type: Number, min: 0, default: 0 },
  available: { type: Boolean, default: true }
}, { timestamps: true }));

// READ all (optional ?search= filters by title)
app.get("/api/books", async (req, res) => {
  const filter = req.query.search
    ? { title: { $regex: req.query.search, $options: "i" } }
    : {};
  res.json(await Book.find(filter).sort({ createdAt: -1 }));
});

// CREATE
app.post("/api/books", async (req, res) => {
  try {
    res.status(201).json(await Book.create(req.body));
  } catch (err) {
    res.status(400).json({ error: err.message });
  }
```

File Name: server/server.js (continued)

```
});

// UPDATE
app.put("/api/books/:id", async (req, res) => {
  const book = await Book.findByIdAndUpdate(req.params.id, req.body,
    { new: true, runValidators: true });
  if (!book) return res.status(404).json({ error: "Book not found" });
  res.json(book);
});

// DELETE
app.delete("/api/books/:id", async (req, res) => {
  const book = await Book.findByIdAndDelete(req.params.id);
  if (!book) return res.status(404).json({ error: "Book not found" });
  res.json({ message: "Book deleted" });
});

app.listen(5000, () => console.log("Server running on http://localhost:5000"));
```

File Name: client/src/App.jsx

```
import { useState, useEffect } from "react";

const API = "http://localhost:5000/api/books";
const empty = { title: "", author: "", price: "" };

function App() {
  const [books, setBooks] = useState([]);
  const [form, setForm] = useState(empty);
  const [editId, setEditId] = useState(null);
  const [search, setSearch] = useState("");
  const [error, setError] = useState("");

  const load = async () => {
    try {
      const res = await fetch(`${API}?search=${encodeURIComponent(search)}`);
      setBooks(await res.json());
      setError("");
    } catch {
      setError("Cannot reach the server. Is it running on port 5000?");
    }
  };

  useEffect(() => { load(); }, [search]);

  const save = async (e) => {
    e.preventDefault();
    const res = await fetch(editId ? `${API}/${editId}` : API, {
      method: editId ? "PUT" : "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ ...form, price: Number(form.price) || 0 })
    });
    if (!res.ok) { setError((await res.json()).error); return; }
    setForm(empty);
    setEditId(null);
```

File Name: client/src/App.jsx (continued)

```
    load();
  };

  const toggle = async (b) => {
    await fetch(`${API}/${b._id}`, {
      method: "PUT",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ available: !b.available })
    });
    load();
  };

  const remove = async (id) => {
    await fetch(`${API}/${id}`, { method: "DELETE" });
    load();
  };

  const edit = (b) => {
    setEditId(b._id);
    setForm({ title: b.title, author: b.author, price: b.price });
  };

  return (
    <div style={{ fontFamily: "Arial", maxWidth: 720, margin: "20px auto" }}>
      <h1>Library Book Manager</h1>
      {error && <p style={{ color: "crimson" }}>{error}</p>}

      <input placeholder="Search by title..." value={search}
        onChange={e => setSearch(e.target.value)} style={{ width: "100%", padding: 8
            }} />

      <form onSubmit={save} style={{ margin: "15px 0", display: "flex", gap: 6 }}>
        <input placeholder="Title" required value={form.title}
          onChange={e => setForm({ ...form, title: e.target.value })} />
```

File Name: client/src/App.jsx (continued)

```
        <input placeholder="Author" required value={form.author}
          onChange={e => setForm({ ...form, author: e.target.value })} />
        <input placeholder="Price" type="number" value={form.price}
          onChange={e => setForm({ ...form, price: e.target.value })} />
        <button type="submit">{editId ? "Update" : "Add"}</button>
      </form>

      <table border="1" cellPadding="8" width="100%">
        <thead>
          <tr><th>Title</th><th>Author</th><th>Price</th><th>Status</th><th>Actions</t
              h></tr>
        </thead>
        <tbody>
          {books.map(b => (
            <tr key={b._id}>
              <td>{b.title}</td>
              <td>{b.author}</td>
              <td>{b.price}</td>
              <td>{b.available ? "Available" : "Issued"}</td>
              <td>
                <button onClick={() => toggle(b)}>{b.available ? "Issue" :
                    "Return"}</button>
                <button onClick={() => edit(b)}>Edit</button>
                <button onClick={() => remove(b._id)}>Delete</button>
              </td>
            </tr>
          ))}
          {books.length === 0 && <tr><td colSpan="5">No books found</td></tr>}
        </tbody>
      </table>
      <p>Total books shown: {books.length}</p>
    </div>
  );
}
```

File Name: client/src/App.jsx (continued)

```
export default App;
```

File Name: client/src/main.jsx

```
import { createRoot } from "react-dom/client";
import App from "./App.jsx";

createRoot(document.getElementById("root")).render(<App />);
```

## OUTPUT:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS                    + ⌄ bash ... ^ ×

$ node server.js
Server running on http://localhost:5000
MongoDB connected
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS                    + ⌄ bash ... ^ ×

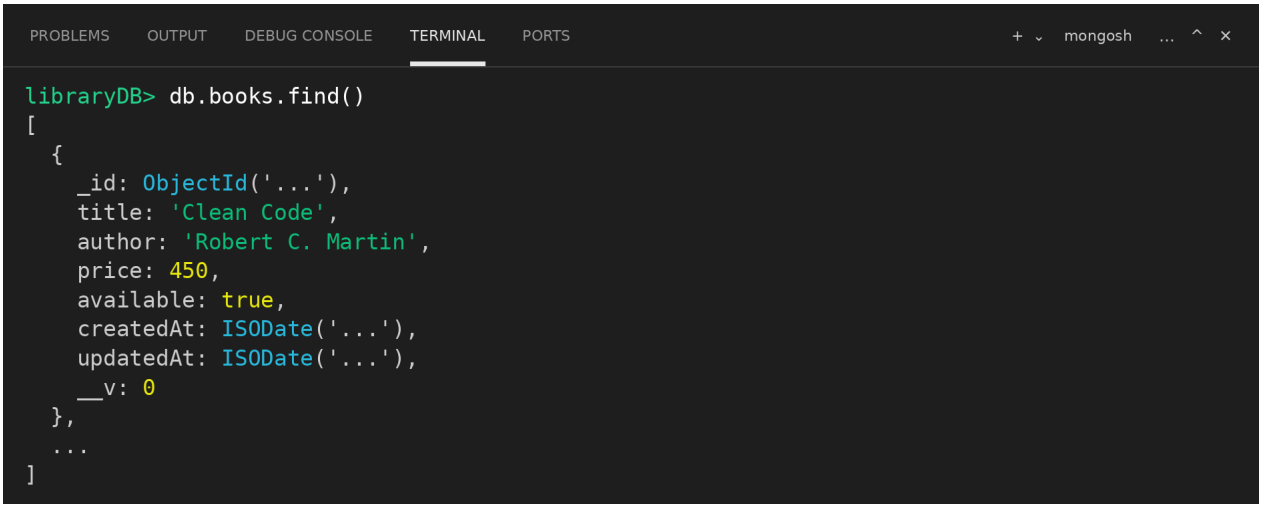
$ npm run dev

  ->  Local:   http://localhost:5173/
  ->  Network: use --host to expose
```

Behaviour of the application at http://localhost:5173:

| Action performed | What the page displays |
|---|---|
| Page opens with an empty database | Table shows "No books found"; Total books shown: 0 |
| Add "Clean Code", "Robert C. Martin", 450 | New row appears with Status Available; Total books shown: 1 |
| Add two more books | The newest book appears at the top; Total books shown: 3 |
| Type "clean" in the search box | Only the Clean Code row is listed; Total books shown: 1 |
| Click Issue on a row | Status changes to Issued and the button changes to Return |
| Click Edit, change the price, click Update | The row shows the new price and the form returns to Add mode |
| Click Delete on a row | The row disappears from the table |
| Stop the server and reload | Red message: Cannot reach the server. Is it running on port 5000? |

Records stored in MongoDB after adding the books, checked in mongosh:




## RESULT:

A full-stack Library Book Manager was developed using Express, React and MongoDB.

## VIVA QUESTIONS:

**Q1. Why is a separate server needed between React and MongoDB?**

Ans: The browser must not connect to the database directly; the server holds the database access and applies validation and rules.

**Q2. Why does `useEffect` depend on `[search]`?**

Ans: So the book list is fetched again every time the search text changes, keeping the table in step with the box.

**Q3. What does `runValidators: true` do in `findByIdAndUpdate`?**

Ans: It applies the schema rules (such as `min: 0`) to updates; by default Mongoose skips validation on update queries.

**Q4. What does `timestamps: true` add?**

Ans: Two fields, `createdAt` and `updatedAt`, which Mongoose fills in automatically.

# FINAL CONCLUSION

The Full-Stack Development Laboratory took me step by step through every layer of a modern web application. In Experiments 1 to 4 (CO1) I built a student profile page with plain HTML, a responsive cafe page with HTML5 and CSS3 variables, Grid, Flexbox and media queries, a marks-analysis program with JavaScript arrays, recursion and higher-order methods, and an interactive task board that updates the DOM through events.

In Experiments 5 and 6 (CO2) I moved to the server side. Writing a custom Node.js module and using the path, fs, os and events modules showed me how code is split into reusable units and why file operations are asynchronous. The Express library API taught me routing, middleware, query strings, validation and the correct use of the status codes 200, 201, 400 and 404.

Experiment 7 (CO3) introduced React, where a profile card, a like button and a stopwatch showed props, state, list rendering and lifecycle cleanup in functional and class components. Experiment 8 (CO3) used JSON Server to practise GET, POST, PUT, PATCH and DELETE, together with filtering and sorting, and made the difference between replacing and partially updating a record clear.

Experiments 9 and 10 (CO4) dealt with the data layer and the complete application. I wrote MongoDB operations in mongosh and in Node.js with Mongoose, including operators, projection, aggregation and schema validation, and then combined Express, React and MongoDB into a Library Book Manager with search, issue and return, edit and delete.

This laboratory has given me a solid base in front-end design, server-side programming, component-based interfaces and database-driven applications, and I will use these skills in my future coursework and projects. I am grateful to my faculty guide for her guidance throughout the course.